Error Catalog: Failure Modes, Detection, and Resolution

Purpose

This document catalogs every anticipated failure mode in the simulation, how to detect it, and what the orchestrator does about it. It is organized by the phase in which the failure can occur.

Each entry is a specification: the code must implement the detection and resolution exactly as described. Entries marked HALT stop the run for debugging; all others are recoverable.

Category 1: LLM Output Failures

These apply to ALL agents (firms, environment, financial institutions).

1.1 Invalid JSON

Detection: json.loads() raises JSONDecodeError.

Resolution (3-stage):

- Extract: Search response for first {...} block using brace counting.

Try parsing that substring.

- Re-prompt: Send the agent a follow-up message:

"Your response was not valid JSON. The parse error was: [error]. Please output ONLY a JSON object with no other text." Include the original context (not truncated).

- Fallback: If 2 re-prompts fail, use the agent-type-specific fallback

(see Section 2).

Logging: Log the raw response, the parse error, and which stage resolved it.

1.2 JSON Valid but Missing Required Fields

Detection: Validate against the agent's output JSON schema. Missing required fields detected.

Resolution:

- Re-prompt: "Your response is missing required field(s): [list].

Please include all required fields."

- Fill defaults: If re-prompt also fails, fill missing fields with

type-specific defaults:

  - Firm: price=last_quarter, production=last_quarter*0.9, all spending=0
  - Environment: use deterministic logit baseline for demand/shares
  - Financial: hold all terms unchanged from last quarter

Logging: Log which fields were missing and what defaults were used.

1.3 JSON Valid but Values Out of Range

Detection: Schema validation or explicit bounds checks.

Field	Valid Range	Clamp To

price	>= 0	max(0, value)
production	[0, capacity]	min(capacity, max(0, value))
capex, rd_spend, sga_spend	>= 0	max(0, value)
dividends, buybacks	>= 0	max(0, value)
rd_allocation values	[0, 1] each, sum ~= 1	Renormalize
market_share (env)	[0, 0.60]	Clamp and redistribute
units_sold (env)	[0, production_i]	min(production_i, value)
total_demand (env)	[0.3x, 3.0x baseline]	Clamp to range
equity_price	> 0 for living firms	max(0.01, value)
rates (bank)	[0, 0.50] per quarter	Clamp

Resolution: Clamp silently and log. Do NOT re-prompt for range violations -- clamping is cheaper and more reliable.

1.4 LLM Timeout

Detection: HTTP request to agent server exceeds 180 seconds.

Resolution:

- Retry once with the same prompt.
- If second attempt also times out: use fallback decision.
- Log the timeout with agent ID, phase, and quarter.

Note: If an agent consistently times out (3+ quarters in a row), log a WARNING suggesting the model may be too slow for the configured hardware.

1.5 Agent Server Unreachable

Detection: HTTP connection refused or DNS failure on health check (run before each quarter) or on turn request.

Resolution:

- Health check before quarter: If agent is unreachable, log ERROR and

use fallback for all that agent's turns this quarter.

- Mid-quarter: Same -- fallback for remaining turns.
- If the agent is the environment: HALT the run. The simulation cannot

proceed without market resolution (even with the deterministic fallback, no narrative is generated).

- If a firm agent: that firm uses fallback decisions; other agents proceed.
- If a financial agent: hold terms unchanged from last quarter.

1.6 Agent Returns Non-JSON Response (Markdown, Prose, etc.)

Detection: No { found in response, or first { extraction yields invalid JSON.

Resolution: Same 3-stage pipeline as 1.1. Common with smaller models that ignore the "output JSON only" instruction.

Prevention: Include a JSON example in the prompt. Use structured output mode if the LLM backend supports it. Keep system prompts short for small models.

Category 2: Agent-Type Fallback Decisions

When all LLM attempts fail, the orchestrator substitutes a deterministic fallback. These are deliberately conservative -- they keep the simulation running but do not make strategic choices.

2.1 Firm Fallback

```
def firm_fallback(firm: FirmState, last_decisions: Decisions) -> Decisions:
    return Decisions(
        price=last_decisions.price,           # hold price
        production=int(firm.capacity_units * 0.7),  # 70% of capacity
        capex=0,                               # no investment
        rd_spend=params.mandatory_phase3_cost, # mandatory only
        rd_allocation={"product": 0.6, "process": 0.3, "delivery": 0.1},
        sga_spend=max(0, last_decisions.sga_spend * 0.5),  # halve marketing
        equity_issuance_request=0,
        debt_request=0,
        dividends=0,
        buybacks=0,
    )
```

Rationale: Hold price, reduce everything else. This is "the firm is coasting" -- not optimal, but not suicidal.

2.2 Environment Fallback

Use the deterministic multinomial logit demand model. No events. No narrative.

```
def environment_fallback(firm_actions, macro, params) -> EnvironmentOutcome:
    total_demand, shares = multinomial_logit_demand(firm_actions, macro, params)
    return EnvironmentOutcome(
        total_demand=total_demand,
        demand_rationale="Deterministic fallback (environment agent unavailable)",
        firm_outcomes=[
            FirmOutcome(
                firm_id=f.firm_id,
                units_sold=min(int(total_demand * shares[f.firm_id]), f.production),
                market_share=shares[f.firm_id],
            ) for f in firm_actions
        ],
        rd_outcomes=[
            RDOutcome(
                firm_id=f.firm_id,
                product_advance=False,
                process_cogs_reduction_pct=0.0,
                delivery_advance=False,
            ) for f in firm_actions
        ],
        events=[],
        narrative="[Automated: environment agent unavailable this quarter. "
                  "Market outcomes determined by baseline demand model.]",
    )
```

2.3 Equity Market Fallback

Hold equity prices unchanged. Reject all new offerings.

```
def equity_market_fallback(last_prices: dict) -> EquityMarketDecision:
    return EquityMarketDecision(
        equity_prices=[
```

```
            {"firm_id": fid, "price_per_share": p, "reasoning": "Fallback: hold"}
            for fid, p in last_prices.items()
        ],
        subscription_decisions=[],
        market_sentiment="neutral",
    )
```

2.4 Investment Bank Fallback

No advisory output. No research.

2.5 Commercial Bank / Credit Fund Fallback

Hold all terms unchanged from last quarter.

```
def bank_fallback(last_terms: dict) -> BankDecision:
    return BankDecision(
        terms=[
            {**t, "reasoning": "Fallback: hold terms unchanged"}
            for t in last_terms
        ]
    )
```

Category 3: Accounting and Invariant Failures

3.1 Balance Sheet Doesn't Balance

Detection: abs(atq - ltq - ceqq) > 1.0 after Phase 6 postings.

Resolution: HALT. This is a bug in the accounting code, not a recoverable error. Dump full state for debugging:

- Prior state
- Decisions (raw and clamped)
- Outcomes
- All intermediate calculations
- The specific posting that broke the identity

Prevention: Pure-function accounting with the doc 16 worked example as a regression test. If this invariant fails, the code is wrong.

3.2 Cash Reconciliation Fails

Detection: abs(chechq - (oancfq + ivncfq + fincfq)) > 1.0

Resolution: HALT. Same as 3.1 -- this is a code bug.

3.3 Retained Earnings Roll-Forward Fails

Detection: abs(RE_end - RE_start - NI + DIV) > 1.0

Resolution: HALT. Code bug.

3.4 Negative Total Assets (Non-Default Firm)

Detection: atq < 0 for a firm that is not in default status.

Resolution: HALT. This should be impossible after correct clamping and settlement. If it occurs, either clamping or settlement has a bug.

3.5 Negative Cash (Non-Default Firm, Post-Settlement)

Detection: cheq < -1.0 after Phase 8 settlement, for a firm not flagged for default.

Resolution: This means settlement failed to catch an insolvency. The firm should have defaulted. Fix: retroactively flag the firm for default and run the bankruptcy process. Log a WARNING -- settlement logic has a gap.

3.6 Current Liabilities Decomposition Fails

Detection: abs(lctq - apq - acoq - txpq - dlcq) > 1.0

Resolution: HALT. Posting bug.

3.7 Inventory Continuity Fails

Detection: abs(inv_end - inv_start - production_cost + cogs) > 1.0

Resolution: HALT. FIFO calculation bug.

Category 4: Environment Outcome Failures

4.1 Market Shares Don't Sum to 1.0

Detection: abs(sum(market_shares) - 1.0) > 0.01

Resolution:

- Small deviation (0.01-0.05): Renormalize silently. Log.
- Large deviation (> 0.05): Re-prompt environment with specific error.
- After 2 re-prompts: Use deterministic fallback (2.2).

4.2 Units Sold Exceed Production

Detection: units_sold_i > production_i for any firm.

Resolution: Clamp units_sold_i = production_i. Redistribute excess units proportionally to other firms (up to their production). Log.

4.3 Units Sold Don't Sum to Total Demand

Detection: abs(sum(units_sold) - total_demand) > 1

Resolution: Adjust the largest firm's units_sold to make the sum exact. Log.

4.4 Total Demand Out of Bounds

Detection: total_demand < 0.3 baseline or total_demand > 3.0 baseline

Resolution:

- Re-prompt: "Your total demand of [X] is outside the expected range [0.3baseline, 3.0baseline]. Please reconsider."
- After 2 re-prompts: Clamp to the nearest bound. Log WARNING.

4.5 R&D Advance Granted Prematurely

Detection: Environment says product_advance: true for a firm with cumulative product R&D below the minimum threshold.

Resolution: Override to product_advance: false. Log WARNING. The orchestrator enforces R&D thresholds -- the environment agent cannot bypass them.

4.6 Event Spam (Too Many Events)

Detection: More than 2 events in a single quarter, or events in more than 50% of the last 8 quarters.

Resolution:

- Keep the first event, discard the rest. Log.
- If persistent (event spam in 4+ consecutive quarters): add to the

environment's next prompt: "Note: events should be rare. Most quarters have zero events."

4.7 Narrative Contradicts Outcomes

Detection: Not automated (would require NLI model). Flagged for human review in diagnostics.

Resolution: Log the narrative and outcomes for post-run review. No automated fix. The narrative is not load-bearing -- it is context for agents, not accounting input.

Category 5: Financial Institution Failures

5.1 Equity Price is Zero or Negative for Living Firm

Detection: price <= 0 for a non-default firm.

Resolution: Set price = max(0.01, book_value_per_share * 0.5) as a floor. Log WARNING.

5.2 Equity Price Change > 50% in One Quarter

Detection: abs(P_new / P_old - 1) > 0.50

Resolution: Soft warning only. Do NOT clamp -- large price changes can be justified (e.g., Gen 2 breakthrough, default scare). Log for diagnostics review.

5.3 Revolver Commitment Exceeds Bank Capital Limit

Detection: Single-firm commitment > max_single_exposure_pct * bank_capital.

Resolution: Clamp commitment to the limit. Log.

5.4 Interest Rate is Unreasonable

Detection: Rate < 0 or rate > 0.25 per quarter (100% annual).

Resolution: Clamp to [risk_free_rate, 0.25]. Log.

5.5 Bank Capital Falls Below Minimum

Detection: capital_ratio < min_capital_ratio after posting losses.

Resolution: Apply distress constraints (doc 07):

- Undercapitalized: no new commitments, rate floor

- Critically undercapitalized: no new business
- Failed (capital <= 0): replace institution next quarter

This is handled in Phase 8 (settlement) and is not an error -- it is an expected simulation outcome.

Category 6: Orchestrator Infrastructure Failures

6.1 Checkpoint File Corrupted

Detection: pickle.load() raises exception, or loaded state fails validation.

Resolution: Fall back to previous quarter's checkpoint. If that also fails, fall back to the one before. If 3 consecutive checkpoints are bad, HALT -- something is systematically wrong.

6.2 Compustat Panel Write Fails

Detection: CSV write raises IOError, or post-write validation detects duplicate keys.

Resolution: Retry write. If disk is full, HALT with clear error message. If duplicate keys: the prior quarter was double-posted. Remove the duplicate and re-write.

6.3 SIGINT (User Interrupts Run)

Detection: Signal handler catches SIGINT.

Resolution:

- Complete the current phase (do not interrupt mid-posting).
- Save checkpoint.
- Print resume instructions: "Run interrupted at Q[X]. Resume with:

python -m llm_firm_lab run --resume --run-id [id]"

- Exit cleanly.

6.4 Out of Memory

Detection: MemoryError.

Resolution: This is most likely caused by loading too many past-run Compustat panels into memory for cross-run retrieval. Prevention:

- Lazy-load past runs (don't load all at startup)
- Use SQLite for cross-run queries instead of in-memory DataFrames
- Log memory usage per quarter; warn if > 2GB

6.5 Agent Memory Database Corrupted

Detection: SQLite integrity check fails, or queries return unexpected results.

Resolution: The agent's memory is non-critical for simulation correctness (the orchestrator has the canonical state). If an agent's memory.db is corrupt:

- Log WARNING.
- Create a fresh memory.db for that agent.
- The agent loses its reasoning history but can still operate (it receives

fresh context each quarter from the orchestrator).

Category 7: Diagnostic Warnings (Non-Critical)

These are not errors -- they are patterns that suggest the simulation may be producing unrealistic or degenerate results. Flagged in the diagnostics report.

7.1 All Firms Identical

Detection: Standard deviation of prices, R&D, and SGA across firms is < 5% of the mean for 4+ consecutive quarters.

Implication: Firms are not differentiating. Fingerprints may not be effective, or the prompt is too constraining.

7.2 Zero Investment

Detection: Any firm with capex=0 AND rd_spend=mandatory_only for 8+ consecutive quarters.

Implication: Firm is coasting. Will be left behind on Gen 2 race. May be a strategic choice (conserving cash) or a prompt failure.

7.3 Market Share is Static

Detection: Max change in any firm's market share < 1 percentage point for 8+ consecutive quarters.

Implication: The demand system or environment agent is not responding to price/quality differences. Check if the environment is just copying the baseline allocation.

7.4 Consistent Overvaluation or Undervaluation

Detection: Mean pricing error (P - P*) for the Equity Market agent is > 20% (positive or negative) over 8+ quarters.

Implication: The valuation framework is biased. The Equity Market agent may be systematically optimistic or pessimistic.

7.5 Death Spiral

Detection: A firm slot has had 3+ incarnations default in Q1 (immediately after IPO).

Implication: IPO capitalization is insufficient, or the firm fingerprint is consistently poor. Slot should be paused per doc 10.

7.6 Bank Portfolio Concentration

Detection: Any financial institution has > 40% of its portfolio in a single firm.

Implication: Concentration risk. If that firm defaults, the institution may become distressed. Flag for review.

7.7 Inventory Buildup

Detection: Any firm's inventory exceeds 4 quarters of sales volume.

Implication: Firm is producing more than it can sell. Cash is being locked up in unsold inventory. May indicate a pricing or demand problem.

7.8 Cash Burn Acceleration

Detection: CFO negative and becoming more negative for 4+ consecutive quarters, while revenue is not growing.

Implication: Firm is heading toward default. The financial agents should be tightening credit, and the equity price should be declining.

Summary: Resolution Hierarchy

For any failure during a run, the orchestrator follows this hierarchy:

```
1. FIX SILENTLY (clamp, renormalize, fill default)
   -> Log it, continue the run.

2. RE-PROMPT THE AGENT (up to 2 retries)
   -> If the fix requires the agent to try again.

3. USE FALLBACK DECISION
   -> If re-prompts fail, substitute a conservative default.

4. WARN AND CONTINUE
   -> If the issue is non-critical (diagnostic warning).

5. HALT THE RUN
   -> ONLY for accounting invariant violations (code bugs)
      or unrecoverable infrastructure failures.
```

The goal is: never HALT unless the code itself is broken. LLM failures are expected and recoverable. Market weirdness is expected and self-correcting. Only violations of accounting identity are grounds for stopping.